

Bishwa Thapa

Sr. Site Reliability / DevOps Engineer · thapabishwak@gmail.com

Infrastructure and platform engineer, 8+ years across security SaaS (Cloud Browser Isolation at Zscaler, CNAPP at Plerion, Open XDR / NG-SIEM at Stellar Cyber): today operating an organization's entire production Kubernetes footprint; before that, led a security product line. The repeated move across every role: take operational knowledge trapped inside experts and move the safety into the system itself, so correct operation becomes the default and deep expertise becomes optional rather than required. Built okectl (fleet Kubernetes operations as checkpointed, resumable workflows that paid for themselves in reclaimed cloud spend), Edge Proxy (request-time routing that removed a hard DNS scaling ceiling, still in production years later), and kube-collector-manager (one pull-only in-cluster security agent identical on any cloud). Now building AI agents that do real operations work behind explicit human approval gates.

EXPERIENCE

Sr. SRE / DevOps Engineer · Stellar Cyber

Dec 2024 – Present

Sole fleet-operations team for the production Kubernetes behind Stellar Cyber's Open XDR / NG-SIEM platform: two OCI tenancies (commercial and government), ~100 multi-tenant NGSaaS OKE clusters plus ~50 single-tenant PreSaaS sites (~2,200 nodes) across 15+ OCI regions, serving ~250 customer organizations, with follow-the-sun production on-call.

Flagship: okectl. Go CLI for OKE fleet day-2 operations: every fleet goal (upgrade, evolve, migrate, scale, shrink, cutover) is a single command. Every mutating operation renders a plan, applies with per-phase checkpoints, and resumes from the failed step; sizing and preflight both delegate to the upstream cluster-autoscaler estimator. Now the standard path for fleet operations. **A Q2 2026 reclaim campaign returned 420+ nodes across ~50 clusters, ~19% of the fleet and ~\$0.6M a year of run-rate cloud spend, executed solo in ~10 engineer-hours with zero customer-visible disruption. Upgrade time fell ~60% (1–2h to 15–20 min per cluster), saving ~120 engineer-hours per cycle across 2–3 fleet cycles a year. Fleet operations opened from 2 of 8 engineers to all 10.**

- **Plan / apply / resume mechanics.** Plans are saved, reviewable artifacts; rollback walks reversible phases backward and refuses at irreversible ones (control-plane upgrades, pool deletion); upgrade walks beyond OKE's 3-minor version skew are refused outright, surfacing recovery guidance instead.
- **Shrink safety.** Victim nodes come from the cluster-autoscaler's own eligibility, drainability, and pod-disruption-budget predicates, filtered by a zonal-volume guard and a mission-critical pin list, then re-validated at apply time so apply equals preview; the campaign verified zero stateful data pods touched and zero PV remounts.
- **Guardrails.** Cloned pools inherit security groups, labels, and image from schema-validated specs, with estimator-scored sizing; the misconfigured-pool class manual operations used to produce is prevented by construction.
- **Fleet reliability.** Own a ~320-rule production alert surface backed by 150+ runbooks. Ended a recurring cascade failure class (a starved node's workloads jumping to, and toppling, the next) with kubelet and system resource reservations plus hard eviction thresholds in fleet-wide node bootstrap, noisy workloads corralled onto tainted component pools: node-unreachable pages to zero, kubelet scrape failures down ~93%, crash loops down ~94%, anchoring a ~75% drop in fleet page volume (~130 to ~32 a day) over two quarters.
- **AI-agent customer onboarding.** Cut onboarding from 2–4 hours to 15–30 minutes (~90%) across 31 production orgs: an agent flow with human confirmation gates around an idempotent plan/diff/provision tool. Also cut cluster provisioning from 1–2 weeks to 1–2 days through OCI and Helm provider modernization.
- **FreeLens AI extension.** Multi-agent operations assistant inside the open-source Kubernetes IDE: read-only kubectl and Prometheus for diagnosis, every mutating action behind explicit human approval (a denial tells the agent not to retry), hooks on 28 resource kinds plus discovered CRDs; a built-in data-pipeline cockpit renders the live ingestion topology under ~50 thresholded Prometheus metrics feeding ~25 risk rules with playbook triage steps.

Stealth Venture

May 2024 – Nov 2024

Took the engineering-lead seat on a team of four building an enterprise browser: a Chromium-based managed workspace with security in the browser itself rather than behind remote isolation, targeting gaps observed while building the cloud-isolation class of products at Zscaler. Took the build to prospective customers to test whether it solved a real problem; the product did not find its market, but the engineering went deep into Chromium internals.

- **Native fork security stack.** Built the fork's security features as native Chromium code: screen-capture prevention at the OS compositor level and verified against eight distinct capture paths; encrypted downloads with transparent in-memory decryption; native ad/tracker blocking; enterprise DLP across clipboard, printing, and downloads through Chromium's standard policy pipeline.
- **Omaha update server.** Wrote the Go update server that speaks Chrome's update protocol natively (v3 XML, v3.1/v4 JSON), serving signed CRX3 updates with Ed25519 response signing and per-IP rate limiting: the browser ships updates without Google's infrastructure.

Software Engineering Lead, ShiftLeft Security & KSPM · Plerion

Feb 2023 – Apr 2024

Joined the cloud-security (CNAPP) platform with a one-line executive mandate for an IaC-security product (ShiftLeft) and a Kubernetes-posture product (KSPM): no spec, no roadmap, no team. Built both roadmaps in month one from competitive research, then ShiftLeft itself in ~2 months, in customers' hands within a quarter of joining. Reported to the CTO as de facto architect for both lines, choosing their technology stacks and presenting roadmaps to the CEO. KSPM grew into a customer-facing product, with the platform operating on millions of cloud assets and findings.

- **Product ownership.** Drove pricing strategy and ran regular roadmap and status reviews with stakeholders; plumbed per-service usage metrics to S3 and built the BI dashboards the company used to track each customer's monthly usage units for ShiftLeft and KSPM, the telemetry that usage-based pricing runs on.
- **Sole engineer to a team of three.** Set the architecture and ownership boundaries that let the team ship without routing through one person.

Flagship: kube-collector-manager. The Go in-cluster agent behind KSPM, built on controller-runtime and shipped as one Docker image and Helm chart. The design move: one Kubernetes-native agent identical on AWS, GCP, and Azure instead of three per-cloud scanner builds, with a pull-only security posture. **Captures every resource manifest and scans every running container image; three cloud integrations in one codebase; zero inbound connections; scans fully offline.**

- **Pull-only model.** The agent polls the platform outbound and uploads results through presigned URLs; nothing on the platform side reaches into the customer cluster.
- **Per-request authentication.** Every request signed with HMAC-SHA256, the shared secret never transmitted, replacing the long-lived privileged API key first asked for.
- **Self-contained scanning.** Trivy ships in the image, SHA256-verified at build; scans run offline against a pre-baked vulnerability DB when present. Leader election, Prometheus metrics, and health probes complete the operational surface.
- **Upstream-first.** Built the missing trivy and trivy-kubernetes capabilities upstream rather than maintaining a private fork, keeping the products on stock Trivy.

Sr. DevOps Engineer (promoted Apr 2021 from Software Engineer II), Cloud Browser Isolation · Zscaler

Jul 2019 – Jan 2023

Joined a recently acquired startup team inside Zscaler as its first platform- and infrastructure-focused engineer, early in the product's Kubernetes journey, and grew with it from migration through hyperscale.

Flagship: Edge Proxy. Multi-region request-time routing for the isolation platform, a Go control plane with an OpenResty/Lua data plane; founded the Go control plane and authored the entire Lua request path. The static-DNS scheme it replaced materialized a record set for every enabled region combination per customer, 2^N growth that ran into a hard Route 53 limit; Edge Proxy instead resolves every request in milliseconds to one of three decisions: proxy in the local region, 307 to the closest live region serving that profile, or honor the session-pinned region. **Cut DNS records from ~2 million across ~300 customers to a few hundred and unlocked 20–30x customer headroom; the same core still runs the product today.**

- **Control-plane/data-plane split over Redis.** A leader-elected Go loop scores regions by health and latency and rewrites per-region state through a diff pattern readers never observe half-applied; every request resolves with a single Redis call.
- **Backwards compatibility by reconstruction.** Each region recomputes at runtime the hostname set its DNS records used to encode; every existing client URL kept working.
- **Stickiness and health as routing primitives.** A pinned-region cookie is revalidated against live health, and an unhealthy pinned region falls through to the closest live one; authored the region-stickiness ADR and sequenced its four-phase rollout across three teams' release trains.

- **Platform foundation and cost.** Moved the stack onto EKS on AWS infrastructure built from scratch in Terraform, then ran and tuned it through hyperscale: ~40% off a ~\$100k/month production AWS bill via capacity-modeled autoscaling, right-sizing, Spot, and off-peak shutdowns. Reliability matured to 99.9% uptime across dozens of clusters in 10 AWS regions.
- **Secure file transfer.** Enterprise customers would not exempt *.amazonaws.com from their filtering policies; moved file delivery to a Zscaler domain via CloudFront in front of S3, rebuilt the upload-to-download flow, and cleared the trust blocker. The product's early \$1–2M ARR deals closed soon after.
- **On-call arc.** Carried production on-call alone, 24/7, on joining; raised it and drove the redistribution into a structured team rotation. Onboarded and ramped 2–3 engineers across the stack and trained the official SRE team later formed to take over the pager.

Software Engineer (First On-site Hire) · Vitapio Portal

*Jun 2018 – Jul
2019*

Joined as the first on-site hire; helped relocate R&D on-site, then hire and onboard the founding engineering team.

- **Monolith to microservices.** Re-architected the monolith into microservices and introduced git-based CI/CD and testing practices alongside, lifting delivery quality and throughput ~30%.

TECH STACK

- **Languages:** Go, Python, TypeScript, Lua, C/C++
- **Kubernetes:** OKE, EKS, controller-runtime, cluster-autoscaler internals, Helm
- **Clouds:** OCI, AWS
- **laC:** Terraform, Packer, Ansible
- **Observability & data:** Prometheus, Grafana, Loki, PagerDuty, Elasticsearch, Kafka, Redis, Spark, MongoDB
- **CI/CD:** Jenkins, GitHub Actions, Argo CD
- **AI/LLM:** Agent design patterns (multi-agent supervisor/specialist orchestration, human approval gates), Anthropic SDK

OPEN SOURCE

- **trivy & trivy-kubernetes (Aqua Security).** 10 PRs merged upstream and in use in the project today: QPS/burst rate controls ([trivy #5910](#), [trivy-kubernetes #263](#)), all-namespace scanning ([trivy #5043](#)), owned-resource exclusion ([trivy #5059](#), [trivy-kubernetes #215](#)), managed-fields stripping ([trivy-kubernetes #280](#)).
- **fluent-operator.** [PR #1458](#) adds envFrom support to the Fluentd CRD.